

## launch system-

describe the configuration of their system and then execute it as described.

configuration of the system includes what programs to run, where to run them, what arguments to pass them.

make it easy to reuse components throughout the system by giving them each a different configuration.

Launching multiple nodes and configuring them at runtime is managed by a framework called `launch_ros`, which builds upon the more generic `launch` framework. `launch_ros` extends `launch`.

The `launch` package is a generic, non-ROS-specific framework used for starting and managing processes. Because `launch` is not specific to ROS, `launch_ros` extends it by adding ROS-related functionalities.

`launch_ros` inherits the generic capabilities of `launch` and adds ROS-specific functionalities.

**from launch import LaunchDescription-** hold all launch configurations.

**from launch\_ros.actions import Node**

**def generate\_launch\_description():** return a launchdescription object which contain the configuration of all nodes.

```
    return LaunchDescription([
Node(
    package='turtlesim',
    namespace='turtlesim1',
    executable='turtlesim_node',
    name='sim'
),
Node(
    package='turtlesim',
    namespace='turtlesim2',
    executable='turtlesim_node',
    name='sim'
```

When launching the two `turtlesim` nodes, the only difference between them is their namespace values.

Both turtles in this system receive commands over the same topic and publish their pose over the same topic. With unique namespaces, messages meant for different turtles can be distinguished.

```
Node(
    package='turtlesim',
    executable='mimic',
    name='mimic',
    remappings=[
        ('/input/pose', '/turtlesim1/turtle1/pose'),
        ('/output/cmd_vel', '/turtlesim2/turtle1/cmd_vel'),
```

you can think of this mimic node as a temporary (temp) node that acts as a bridge between the two Turtlesim nodes. it's better to call it an **intermediate node** or a **relay node**.because its main purpose is to take input from one node and send output to another.Doesnt generate data of its own just listen to pose of turtle1 and forwards it to cmd\_vel of turtle2.

```
workspace/src/package/launch/
```

for colcon to recognize launch files presence,entry have to be made in the data\_files=[] of setup.py ...**import os** and from glob **import glob**

```
(os.path.join('share', package_name, 'launch'), glob(os.path.join('launch', '*launch.[pxy][yma]*')))
```

os.path.join- joins multiple path components into single valid path.  
Share is the directory.package\_namee-variable holding the name of workspace.  
Launch-subdirectory which launches files.

**glob is a module in Python that finds files and directories matching a pattern.**It searches **recursively** **\*launch.** → **Matches any filename ending with "launch."** (e.g., mylaunch., testlaunch.).

- [pxy] → Matches **any single** character that is **p, x, or y.**
- [yma] → Matches **any single** character that is **y, m, or a.**
- \* → Matches anything after that (e.g., .py, .yaml, .xml).

## PARENT LAUNCH FILE

This means that **one launch file will start another launch file** and pass **arguments (parameters)** to it. This approach is useful for **modular** and **reusable** launch configurations in ROS 2.

```
from launch_ros.substitutions import FindPackageShare
```

```
PathJoinSubstitution([  
  
    FindPackageShare('launch_tutorial'),  
    'launch',  
    'example_substitutions.launch.py'
```

The FindPackageShare substitution is used to find the path to the launch\_tutorial package. The PathJoinSubstitution substitution is then used to join the path to that package path with the example\_substitutions.launch.py file name.

In ROS 2, a **substitution** in a launch file is a **dynamic value** that gets evaluated at runtime.

Substitutions are commonly used for:

1. **Launch Arguments (LaunchConfiguration)** → Getting arguments from the command line.
2. **File Paths (PathJoinSubstitution)** → Constructing file paths dynamically.

What the code does?

1. Declares 3 launch arguments (`turtlesim_ns`, `use_provided_red`, and `new_background_r`).
2. Launches a `turtlesim_node` in the given namespace.-- A **namespace** in ROS 2 (and ROS in general) is a way to organize nodes, topics, services, and parameters. Think of a namespace like a folder in a file system. When we **launch a node in a namespace**, it means that the **node's resources (like topics, services, parameters)** will be organized under that namespace.
3. Sets the background color to 120 by default.

If the conditions are met (`new_background_r == 200` and `use_provided_red` is `True`), it updates the background color to the value of `new_background_r`.

- The background color change is delayed by 2 seconds via `TimerAction`.

## EVENT HANDLERS

Launch in ROS 2 is a system that executes and manages user-defined processes. It is responsible for monitoring the state of processes it launched, as well as reporting and reacting to changes in the state of those processes. These changes are called events and can be handled by registering an event handler with the launch system.

```
##turtlesim_ns = LaunchConfiguration('turtlesim_ns') use_provided_red =  
LaunchConfiguration('use_provided_red') new_background_r =  
LaunchConfiguration('new_background_r')
```

At the beginning, several launch arguments are declared using `DeclareLaunchArgument`. These arguments allow the user to customize the launch process:

- **turtlesim\_ns**: Namespace for the `turtlesim` node.
- **use\_provided\_red**: A flag indicating whether to use a specific red background color.
- **new\_background\_r**: Specifies the red value of the background color ##  
events:

### 1. OnProcessStart:

- This event is triggered when the `turtlesim_node` starts.
- It will log the message "Turtlesim started, spawning turtle" and execute the `spawn_turtle` process.

**RegisterEventHandler(**

```

        OnProcessStart(target_action=turtlesim_node,
        on_start=[LogInfo(msg='Turtlesim started, spawning turtle'),
        spawn_turtle])
    )

```

## 2. OnProcessIO:

- This event listens for stdout output from the spawn\_turtle process.
- It logs the output of the spawn request.

```

RegisterEventHandler(

```

```

    OnProcessIO(target_action=spawn_turtle, on_stdout=lambda event:
    LogInfo(msg='Spawn request says "{}".format(event.text.decode().strip()))
)

```

## 3. OnExecutionComplete:

- This event is triggered when the spawn\_turtle process completes.
- It logs the message "Spawn finished", changes the background color (change\_background\_r), and sets up a 2-second delay (TimerAction) before checking if the background color needs to be updated based on conditions.

```

RegisterEventHandler( OnExecutionComplete(target_action=spawn_turtle,
on_completion=[LogInfo(msg='Spawn finished'), change_background_r,
TimerAction(period=2.0, actions=[change_background_r_conditioned])]) )

```

```

OnProcessExit:

```

- This event occurs when the turtlesim\_node process exits (when the turtlesim window is closed).
- **OnShutdown:**
- This event is triggered when the launch system is asked to shut down.

## MANAGING LARGE PROJECTS

It explains how to launch multiple interconnected nodes in a robotic application, manage their parameters, and configure them for use with ROS 2.

nodes can have many parameters. Nodes need to be launched and configured with the appropriate parameters. This process is often cumbersome to do manually for each node. Instead, you can use a launch file to automate the process of starting these nodes with specific parameters. Parameters can be sent through launch files.

**Launch files are used to start and configure multiple nodes at once.**

- In the context of this tutorial, the goal is to create a **launch file** (called launch\_turtlesim.launch.py) that:

- **Starts two turtlesim simulations** (representing two turtles).
- **Starts the TF broadcaster and listener** nodes to manage the position and movement of the turtles.
- **Loads parameters** to control the turtles' behavior (e.g., speed, color).
- **Launches an RViz configuration** (RViz is a 3D visualization tool for ROS).

## 1. Top-level organization

A top-level launch file is a higher-level script that organizes and calls multiple lower-level launch files. Improves modularity, reusability.

```
turtlesim_world_1 = IncludeLaunchDescription(
    PythonLaunchDescriptionSource([os.path.join(
        get_package_share_directory('launch_tutorial'), 'launch'),
        '/turtlesim_world_1.launch.py'])
    )

turtlesim_world_2 = IncludeLaunchDescription(

    PythonLaunchDescriptionSource([os.path.join(
        get_package_share_directory('launch_tutorial'), 'launch'),
        '/turtlesim_world_2.launch.py'])
    )
```

This **loads and executes another launch file** within the current launch file.

Broadcaster\_listener\_nodes-launches broadcaster and listener node (explained later) using tf2 for coordinate transformations.

Mimic\_node-launches a node that mimics another turtle's motion.

Fixed\_frame\_node- runs a fixed transform broadcaster which sets static or permanent relationship between two frames.

rviz\_node-Starts **Rviz 2** with a pre-configured display for **visualizing the turtlesim world**.

## 2.Parameters

a) setting parameters

```
background_r_launch_arg = DeclareLaunchArgument(
    'background_r', default_value=TextSubstitution(text='0')
    )

background_g_launch_arg = DeclareLaunchArgument(
    'background_g', default_value=TextSubstitution(text='84')
```

```
)
background_b_launch_arg = DeclareLaunchArgument(
    'background_b', default_value=TextSubstitution(text='122')
)
```

Defines three launch arguments: background\_r, background\_g, and background\_b.  
 These represent the RGB color values for the turtlesim window background.  
 The default values are (0, 84, 122)

```
Node(
    package='turtlesim',
    executable='turtlesim_node',
    name='sim',
    parameters=[{
        'background_r': LaunchConfiguration('background_r'),
        'background_g': LaunchConfiguration('background_g'),
        'background_b': LaunchConfiguration('background_b'),
    }]
)
```

->Launches turtlesim.Passes the **background color parameters** dynamically using LaunchConfiguration().Background color can be changed without modifying the launch file.

#### b) Loading parameters from YAML file

Defining arguments and parameters in YAML files make it easy to store and load a large number of variables.Configuration file

#### c) Using wildcard in YAML

we want to set the same parameters in more than one node. These nodes could have different namespaces or names but still have the same parameters. Wildcard characters, which act as substitutions for unknown characters in a text value, to apply parameters to several different nodes.

Therefore, instead of creating a new configuration for the same node that use the same parameters, we can use wildcards syntax. `/**` will assign all the parameters in every node, despite differences in node names and namespaces.

`/**:`

```
ros__parameters:
  background_b: 255
  background_g: 86
  background_r: 150
```

### 3) Namespaces

Unique namespaces allow the system to start two similar nodes without node name or topic name conflicts. Two namespaces can have nodes of same name but a namespace can't have same named nodes.

If the launch file contains a large number of nodes, defining namespaces for each of them can become tedious. To solve that issue, the `PushRosNamespace` action can be used to define the global namespace for each launch file description, instead of explicitly defining the namespace for each node. `PushRosNamespace` is used to **set a global namespace** for the entire launch description or for nested nodes. When this action is used, all **subsequent nodes** will automatically inherit this namespace

```
def generate_launch_description(): return LaunchDescription([
  PushRosNamespace('robot_namespace')
  Node(
    package='my_package',
    executable='my_node_1',
    name='node_1',
  ),
  Node(
    package='my_package',
    executable='my_node_2',
    name='node_2'
  ),
])
```

Both `node_1` and `node_2` will inherit the `'robot_namespace'`. So, their full names will be:

- /robot\_namespace/node\_1
- /robot\_namespace/node\_2

#### 4) Reusing nodes

There are no namespaces explicitly defined for the nodes. Therefore, by default, all nodes will belong to the root namespace, which is simply / (i.e., no specific namespace is applied).

**def generate\_launch\_description():**

```
    return LaunchDescription([
        DeclareLaunchArgument(
            'target_frame', default_value='turtle1',
            description='Target frame name.'
        ),
```

**Node(**

```
    package='turtle_tf2_py',
    executable='turtle_tf2_listener',
    name='listener',
    parameters=[
        {'target_frame': LaunchConfiguration('target_frame')}
    ]
),
```

In this file, we have declared the `target_frame` launch argument with a default value of `turtle1`. The default value means that the launch file can receive an argument to forward to its nodes, or in case the argument is not provided, it will pass the default value to its nodes.

#### 5) Parameter Overrides

In ROS 2, **parameter overrides** allow you to change the default values of parameters when launching nodes or other launch files. This is particularly useful when you need to modify specific parameters for nodes or launch files without changing the source code or configuration files.

**broadcaster\_listener\_nodes = IncludeLaunchDescription(**

```
    PythonLaunchDescriptionSource([os.path.join(
        get_package_share_directory('launch_tutorial'), 'launch'),
        '/broadcaster_listener.launch.py']),
    launch_arguments={'target_frame': 'carrot1'}.items(),
)
```

Here, you are providing **launch arguments** that will be forwarded to the included launch file. Specifically, you are overriding the `target_frame` parameter by passing `{'target_frame': 'carrot1'}`.

- **target\_frame: 'carrot1':** This means that you are overriding the default value of the `target_frame` argument (which is likely declared with a default value inside `broadcaster_listener.launch.py`) to use 'carrot1' instead of the default.
- the `target_frame` parameter likely has a default value in the `broadcaster_listener.launch.py` file. By overriding it in this launch description, you ensure that the included launch file gets a custom value ('carrot1') for `target_frame` instead of the default value.

## 6) Remapping

### MIMIC NODE

This is a node that receives the pose (position and orientation) of one turtle and uses that information to control the movement of another turtle.

we want it to get this data from a different topic: `/turtle2/pose` (the pose of turtle2 in the initial turtlesim world).

To integrate with the turtlesim2 world, we remap this topic to `/turtlesim2/turtle1/cmd_vel`.

This is helpful if you want to test the same functionality in different environments or with different robots.

## TF2

The `tf2` library in ROS 2 is a powerful tool for managing coordinate frames and transformations between them. **tf2** to manage transformations between three coordinate frames: a world frame, a turtle1 frame, and a turtle2 frame

### World Frame:

This is the global reference frame. All other frames (such as turtle1 and turtle2) are typically described relative to this frame.

### tf2 Broadcaster:

The **tf2 broadcaster** is used to publish the coordinate frames of the turtles. This means that each turtle's position and orientation relative to the world frame is broadcasted over the ROS 2 network. The `tf2` broadcaster continuously sends updates about the transformations (i.e., position and orientation) of the turtles.

### tf2 Listener:

The **tf2 listener** is used to listen to these transformations and compute the difference

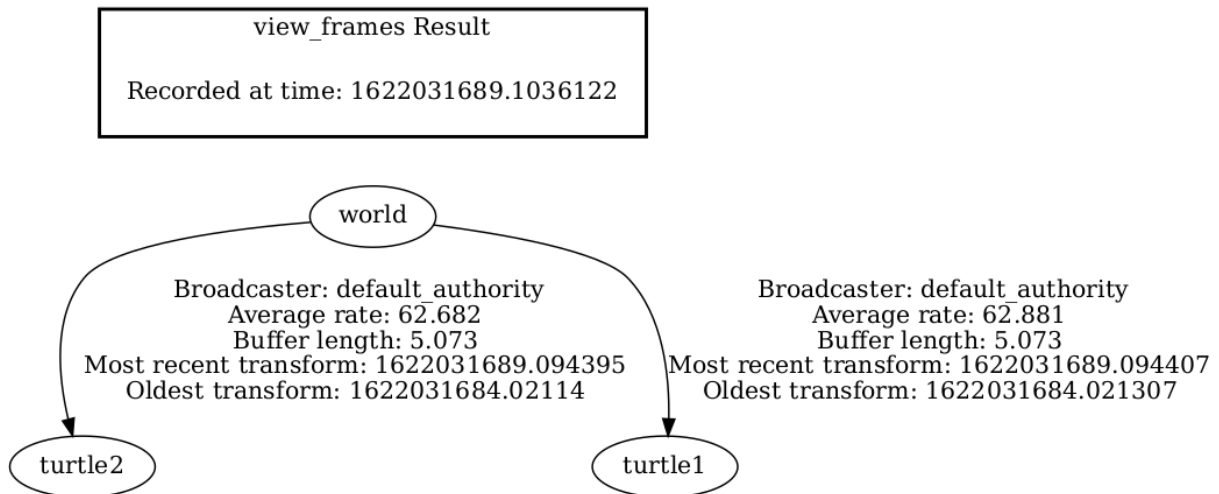
between the two turtle frames. In this case, the listener is interested in the relative position of turtle2 and turtle1 in the world frame for following motion. Essentially, the listener calculates how far turtle2 is from turtle1 and applies that difference to the movement commands of turtle1.

## TF2 TOOLS

### 1)view-frames

view\_frames creates a diagram of the frames being broadcast by tf2 over ROS.

```
ros2 run tf2_tools view_frames .....
```



The world frame is the parent of the turtle1 and turtle2 frames.

## BROADCASTER

### static broadcaster

Used for static transformations that do not change over time.

Once published, the transformation remains valid indefinitely.

### Broadcaster

Used for dynamic transformations that change over time.

Typically used for **moving robots, joints, or objects** whose positions change frequently.

```
def quaternion_from_euler(ai, aj, ak):
```

```
    ai /= 2.0
```

```

    aj /= 2.0
    ak /= 2.0
    ci = math.cos(ai)
    si = math.sin(ai)
    cj = math.cos(aj)
    sj = math.sin(aj)
    ck = math.cos(ak)
    sk = math.sin(ak)
    cc = ci*ck
    cs = ci*sk
    sc = si*ck
    ss = si*sk

    q = np.empty((4, ))
    q[0] = cj*sc - sj*cs #x
    q[1] = cj*ss + sj*cc#y
    q[2] = cj*cs - sj*sc#z
    q[3] = cj*cc + sj*ss#w

    return q

```

The function `quaternion_from_euler(ai, aj, ak)` converts **Euler angles** (roll, pitch, yaw) into a **quaternion**. Roll- about x axis; pitch – about y axis; yaw – about z axis

```

cc = ci * ck
cs = ci * sk
sc = si * ck
ss = si * sk

```

A **quaternion** is a way of representing a 3D rotation without **gimbal lock**. **Gimbal lock** is a problem that occurs when using **Euler angles** to represent **3D rotations**. It happens when two rotation axes become **aligned**, causing the loss of one degree of freedom.

```

def make_transforms(self, transformation):
    t = TransformStamped()

```

```

t.header.stamp = self.get_clock().now().to_msg()
t.header.frame_id = 'world'
t.child_frame_id = transformation[1]

t.transform.translation.x = float(transformation[2])
t.transform.translation.y = float(transformation[3])
t.transform.translation.z = float(transformation[4])
quat = quaternion_from_euler(
    float(transformation[5]), float(transformation[6]), float(transformation[7]))
t.transform.rotation.x = quat[0]
t.transform.rotation.y = quat[1]
t.transform.rotation.z = quat[2]
t.transform.rotation.w = quat[3]

self.tf_static_broadcaster.sendTransform(t)

```

**Make\_transforms-** This function is responsible for **creating and broadcasting a static transform** between a **parent frame (world)** and a **child frame (transformation[1])**

**t = TransformStamped()-** Creates an instance of TransformStamped, which is a **ROS 2 message type** used to define transformations.

A TransformStamped message in ROS 2 (Robot Operating System 2) is a fundamental data structure used to represent the spatial relationship (transformation) between two coordinate frames at a specific point in time.

**Parent frame** (the reference frame)

- **Child frame** (the frame being transformed)
- **Translation** (position)
- **Rotation** (orientation using quaternions)

```

self.target_frame = self.declare_parameter(

    'target_frame', 'turtle1').get_parameter_value().string_value

```

This code is a **ROS 2 node** that listens to a TF2 transform, spawns a second turtle (turtle2), and moves it towards turtle1 in **Turtlesim**.

## LOOKUPTRANSFORM()

lookupTransform is used to **find the transformation (position + orientation)** between two frames at a given time. Each object has its own **coordinate frame**.

You need to know **where one object is relative to another**—this is where lookupTransform helps.

```

t = self.tf_buffer.lookup_transform(

    to_frame_rel,      # The frame we want to transform into (e.g., "turtle2")

    from_frame_rel,    # The frame we are transforming from (e.g., "turtle1")

```

```
rcplpy.time.Time() # Get the latest available transform
```

```
)
```

## Why is Time Used in lookupTransform?

In ROS 2 TF2, transformations are time-dependent because frames move over time. When calling lookupTransform, you need to specify **which point in time** you want the transformation for.

```
<?xml version="1.0"? >
```

URDF- unified robot description format- specifies the geometry and organization of robots.

|                    |                                                                                     |                         |
|--------------------|-------------------------------------------------------------------------------------|-------------------------|
| <b>Link</b>        | Represents a <b>physical part</b> of the robot (like body, arm, or leg).            | base_link,<br>right_leg |
| <b>Joint</b>       | Defines <b>how two links are connected</b> (can be fixed, rotating, sliding, etc.). | base_to_right_leg       |
| <b>Parent Link</b> | The <b>main/base part</b> to which another part is attached.                        | base_link               |
| <b>Child Link</b>  | The <b>attached part</b> that moves or stays connected to the parent.               | right_leg               |

In robot kinematics, every joint connects two links:

- **Parent Link:** The main/base part.
- **Child Link:** The attached/moving part.

```
<link name="right_leg">
```

```
<visual>
```

```
<geometry>
```

```
<box size="0.6 0.1 0.2"/>
```

```
</geometry>
```

```
<origin rpy="0 1.57075 0" xyz="0 0 -0.3"/>
```

```
</visual>
```

```
</link>
```

we gave rpy as  $\pi/2$  so in the pitch(y axis) so it will be parallel to the z axis. And

```
<joint name="base_to_right_leg" type="fixed">
```

```
<parent link="base_link"/>
```

```
<child link="right_leg"/>
```

```
<origin xyz="0 -0.22 0.25"/>
```

```
</joint>
```

```
</robot>
```

## MESHES

Meshes are 3D models used to visually represent parts of a robot in URDF (Unified Robot Description Format).

PR2 is a famous **research robot**, and its **3D meshes** (detailed visual models) have been **reused** in your URDF file. Instead of creating new models, you are using the existing PR2 meshes.

STL DAE

## COLLISION

Collision geometry is the simplified shape used to detect collisions between objects in a simulation or physics engine. It is part of the `<collision>` element in URDF (Unified Robot Description Format). Unlike the visual geometry, which defines how the robot looks, collision geometry defines how the robot physically interacts with its environment. Uses a **box** instead of a complex shape for physics simulation. `<visual>` element, which only affects how the robot looks in visualization tools like **Rviz**

### Quicker Processing (Simplified Collisions)

- **Problem:** Collision detection with complex **3D meshes** is computationally expensive.
- **Solution:** Use **simpler shapes** (box, cylinder, sphere) for **collision detection**, while keeping detailed meshes for visualization.
- **Example:** A robot arm with a complex mesh could have **box-shaped collision boundaries** instead.

### Safe Zones (Enlarged Collisions)

- **Problem:** Some parts of the robot are sensitive (e.g., a fragile sensor or a camera).
- **Solution:** Define a **larger collision zone** around them to prevent other objects from getting too close.
- **Example:** If we want to **protect R2D2's head**, we can define an **invisible larger cylinder** around it.

## INERTIA

The `<inertia>` tag in URDF (Unified Robot Description Format) defines the mass distribution of a robot link. It plays a crucial role in physics simulations by specifying how the mass is distributed around the link's center of mass. This affects how the robot moves and reacts to forces.

`<collision>`

```

    <geometry>
      <cylinder length="0.6" radius="0.2"/>
    </geometry>
  </collision>
  <inertial>
    <mass value="10"/>
    <inertia ixx="1e-3" ixy="0.0" ixz="0.0" iyy="1e-3" iyz="0.0" izz="1e-3"/>
  </inertial>
</link>

```

The **inertia tensor** describes how mass is distributed in an object. It determines how the object resists rotational motion.

| Moment | Description                                |
|--------|--------------------------------------------|
| ixx    | Moment of inertia around the <b>X-axis</b> |
| iyx    | Moment of inertia around the <b>Y-axis</b> |
| izz    | Moment of inertia around the <b>Z-axis</b> |
| ixy    | Product of inertia in <b>XY plane</b>      |
| ixz    | Product of inertia in <b>XZ plane</b>      |
| iyz    | Product of inertia in <b>YZ plane</b>      |

# Understanding Contact Coefficients in Simulation

In robot simulation (e.g., **Gazebo with URDF/SDF**), when two **links** come into contact, their **physical interaction** (e.g., friction, stiffness, damping) needs to be **realistically modeled**. This is done using the `<contact_coefficients>` tag inside the `<collision>` element.

## **mu (Friction Coefficient)**

Defines how much **resistance** an object has **against sliding** when in contact with another object.

## **kp (Stiffness Coefficient)**

Represents how **rigid** the contact surface is. If **kp** is too low, objects may "sink" into each other.

## **kd (Damping Coefficient)**

Controls how much **energy is lost** when two objects collide.

# XACRO

[xacro](#) is a macro language for XML. The xacro program runs all of the macros and outputs the result

xacro allows you to specify properties which act as constants

```
<xacro:property name="width" value="0.2" />
```

You can build up arbitrarily complex expressions in the `${}` construct using the four basic operations (+, -, \*, /), the unary minus, and parenthesis. Examples:

```
<cylinder radius="${wheeldiam/2}" length="0.1"/>
```

```
<origin xyz="${reflect*(width+.02)} 0 0.25" />
```

You can also parameterize macros so that they don't generate the same exact text every time. A section of xml code that can be passed as a parameter to macro allowing to dynamically insert a whole block of elements within another part of your robot description.

## **PUBLISH A STATE**

"Publishing the state in URDF" means using a ros node to broadcast the current 3d pose and config of a robot described in a urdf file to ros system.

The "state of the robot" refers to a complete and consistent description of the robot's current condition, encompassing all relevant information needed to understand its configuration and how it's interacting with its environment.

Joint states are a fundamental concept in robotics that describe the configuration of a robot's joints at a specific point in time.

**robot\_state\_publisher:** This node is responsible for taking the robot's URDF (Unified Robot Description Format) and publishing the joint states and transform information needed for visualizing the robot in tools like RViz. It takes the URDF as input and uses the current joint positions (which it might receive on a topic) to calculate and publish the transformations between the robot's links.

**state\_publisher:** This is a custom node (from the `urdf_tutorial_r2d2` package) likely used to publish the joint states of the robot. It's what provides the joint position information that the `robot_state_publisher` needs. The name suggests it publishes the "state" of the robot, which in this context probably refers to the joint positions. These transformations are then published on the `/tf` topic (or `/tf_static` for static transforms).

Imagine you have a 3D model of a robot (the URDF). The `robot_state_publisher` is like the animator that brings that model to life.

But the animator needs to know how to move the joints of the model. That's where the `state_publisher` comes in. It provides the instructions on how each joint should be positioned.

To get the robot to be interactive (with you and ROS), we need to specify two things: **Plugins and Controllers**.

## GAZEBO

Gazebo can't find the 3D mesh files that represent the visual appearance of some parts of your robot.

In 3D graphics, meshes are collections of vertices, edges, and faces that define the shape of a 3D object. They're like the skin or surface of the object. In Gazebo, meshes are used to visually represent the different parts of your robot. Without meshes, you'll see the underlying kinematic structure (links and joints), but not the actual shape of the robot.

To get ROS 2 to interact with Gazebo, we have to dynamically link to the ROS library that will tell Gazebo what to do. Theoretically, this allows for other Robot Operating Systems to interact with Gazebo in a generic way. In practice, its just ROS.

```
<ros2_control name="GazeboSystem" type="system">
  <hardware>
    <plugin>gazebo_ros2_control/GazeboSystem</plugin>
  </hardware>
  <joint name="head_swivel" />
</ros2_control>

<gazebo>
  <plugin filename="libgazebo_ros2_control.so" name="gazebo_ros2_control">
    <parameters>$(find urdf_sim_tutorial)/config/09a-minimal.yaml</parameters>
  </plugin>
</gazebo>
```

**<ros2\_control> tag:** This tag within the URDF defines a ROS 2 control system

we use the `ros2_control` framework to control robots. It provides:

- **Hardware interfaces** to interact with robot hardware or simulators.
- **Controllers** to command joint positions, velocities, or efforts.

**<hardware>:** This section describes the hardware interface that the control system will use

**<plugin>:** This is where you specify the plugin that will handle the communication between ROS 2 and Gazebo. `gazebo_ros2_control/GazeboSystem` is the standard plugin for this purpose.

**<gazebo> tag with plugin:** This tag is also within the URDF, but it's specifically for Gazebo. It loads the same Gazebo ROS 2 control plugin *within* Gazebo. This plugin acts as the bridge between Gazebo and ROS 2.

**<parameters>:** This is crucial! It specifies a YAML file (`09a-minimal.yaml` in your example) that contains the configuration for the

controllers (like PID gains, joint names, etc.). This file tells the plugin which joints to control and how.

## Need for plugins?

**Passing commands from ROS 2 to Gazebo:** ROS 2 controllers (which you'll define in the YAML configuration file) send commands (e.g., desired joint positions or velocities) to the Gazebo plugin.

**Getting sensor data from Gazebo to ROS 2:** The Gazebo plugin reads sensor data (e.g., joint positions, velocities, forces) from the simulated robot in Gazebo and publishes it as ROS 2 messages.

## How it works:

1. You define the robot's structure and properties in the URDF, including the `<ros2_control>` and `<gazebo>` tags with the plugins and their configuration file.
2. When you launch your simulation, Gazebo loads the URDF, including the Gazebo ROS 2 control plugin.
3. The ROS 2 control system is also initialized, using the configuration from the YAML file.
4. The Gazebo plugin and the ROS 2 control system connect to each other.
5. ROS 2 controllers send commands to the Gazebo plugin.
6. The Gazebo plugin translates these commands into actions within the Gazebo simulation.
7. The Gazebo plugin reads sensor data from the simulation and publishes it as ROS 2 messages.
8. ROS 2 controllers receive the sensor data and use it to update their control loops.

## SPAWNING CONTROLLERS

You've successfully linked ROS 2 and Gazebo and configured a `joint_state_broadcaster`. The reason you're getting **empty joint state messages** is that the `joint_state_broadcaster` needs to know *which joints* to broadcast.

The `joint_state_broadcaster` is a standard ROS 2 **controller**. Its job is to take joint state information from the simulated environment (Gazebo in

this case) and publish it on the `/joint_states` topic as `sensor_msgs/msg/JointState` messages.

The YAML file you showed (controller\_manager section) is *not* enough to fully configure the joint\_state\_broadcaster. it's missing the essential part: the list of joints to broadcast.

## Adding command\_interface and state\_interface Tags

```
<joint name="head_swivel">
  <command_interface name="position" />
  <command_interface name="velocity" />
  <state_interface name="position"/>
  <state_interface name="velocity"/>
</joint>
```

command\_interface: These tags declare what *types* of commands this joint can accept. In this case, the head\_swivel joint can accept both position and velocity commands

that the head joint is correctly displayed in RViz and is being published in the joint\_states topic, the next step is to add a controller

Gazebo interacts with ros2\_control through the gazebo\_ros2\_control plugin.

## Difference Between State Interfaces and Command Interfaces in ros2\_control

State Interfaces (Read-Only)- Used to observe the current state, Provide data such as position, velocity, and effort. Controllers read these values but **cannot modify them**. Updated by hardware interfaces or simulator.

Command Interfaces (Write-Only)- Used to send commands, Controllers write values to move joints by setting desired position, velocity, or effort. hardware interface or simulator reads these values and executes the motion

Command interfaces are used in:

- Position controllers (JointGroupPositionController).
- Velocity controllers (JointGroupVelocityController).
- Effort controllers (e.g., EffortController).

## SETTING UP THE HEAD CONTROLLER

A new controller named head\_controller is added under controller\_manager. It is a position controller and controls the head\_swivel joint. The **command interface is position**, meaning we send position values to move it.

controller\_manager:

ros\_\_parameters:

head\_controller:

type: position\_controllers/JointGroupPositionController

head\_controller:

ros\_\_parameters:

joints:

- head\_swivel

interface\_name: position

## MOVING THE WHEELS

diff\_drive\_base\_controller:

ros\_\_parameters:

publish\_rate: 50.0

`left_wheel_names: [left_front_wheel_joint, left_back_wheel_joint]`

`right_wheel_names: [right_front_wheel_joint, right_back_wheel_joint]`

DiffDriveController which subscribes to a standard Twist cmd\_vel message and moves the robot accordingly. In addition to loading the above configuration, this also opens the !RobotSteering panel,

## RVIZ

### Displays

A "display" is indeed something that visualizes data or objects within the 3D world. It's a way to represent information graphically so that it can be understood by a human observer.

Each display gets its own list of properties. Each display gets its own status to help let you know if everything is OK or not.

**A config file in RViz acts like a blueprint or a save state for your visualization setup.** It's a file (usually with the extension ".rviz") that stores all the details of how you've configured RViz. This includes:

**Displays:** Which types of displays you've added (e.g., RobotModel, LaserScan, Image), what topics they're listening to, and their visual properties (colors, sizes, etc.).

**Tools:** Which tools are active (e.g., Select, Move Camera, 2D Nav Goal) and any specific settings for those tools.

**Views:** How you're looking at the 3D world (camera position, zoom level, perspective).

**Layout:** How the different panels and displays are arranged in the RViz window.

### ORBITAL CAMERA

The orbital camera simply rotates around a focal point, while always looking at that point. The focal point is visualized as a small disc while you're moving the camera:

left mouse: click and drag to rotate.

Middle mouse: it will move upward or downward and clicking on the focal point , it will stay under your mouse.

Right mouse button: Click and drag to zoom in/out of the focal point. Dragging up zooms in, down zooms out.

Scrollwheel: Zoom in/out of the focal point

## **FIRST PERSON CAMERA**

Rotates as you are looking with your head.

Left: click/drag to rotate. control/click to pick the object and look directly at it .

Middle: click/drag to move along

Right: click/drag to move forward and backward.

Scroolwheel: move forward/backward

## **TOP DOWN ORTHOGRAPHIC**

Looks down along the z-axis. And orthographic so farther object doesn't get smaller.

Left: click/drag rotate around z axis

middle: click/drag move along xy plane

right: zoom in/out

scrollwheel: zoom in/out

## **XY ORBIT**

The XY Orbit camera in RViz is very similar to the standard Orbital camera, but with a crucial restriction: the focal point is *constrained to the XY plane*. This means that while you can still rotate the camera around the focal point and zoom in/out, you *cannot* move the focal point

up or down (along the Z-axis). The focal point's Z-coordinate remains constant.

### **Controls (Same as Orbital Camera):**

- **Left Mouse Button:** Rotate around the focal point (which is constrained to the XY plane).
- **Middle Mouse Button:** Move the focal point *within* the XY plane. You won't be able to move it vertically.
- **Right Mouse Button:** Zoom in/out.
- **Scroll Wheel:** Zoom in/out.

### **THIRD PERSON FOLLOWER**

The camera is positioned behind and slightly above the target object . As the target moves, the camera adjusts its position to maintain the specified distance and angle.

## **CO-ORDINATE FRAMES**

The target frame and the fixed frame. The fixed frame is the reference frame used to denote the world frame. This is usually the map, or world, or something similar, but can also be, for example, your odometry frame.

odom is a local coordinate frame used for tracking short-term movement.

The target frame is a reference frame that defines the position and orientation of the camera in relation to a specific object or environment.

**Map Frame:**map is a fixed, global coordinate frame.

- If the target frame is set to the map (or world frame), the camera will provide a view of the entire environment, showing the robot moving around the map.

**Robot Base Frame:**

- If the target frame is set to the base of the robot, the camera will remain fixed relative to the robot's base. In this scenario, as the robot moves, the camera will stay in the same position, and the environment will appear to move around the robot.

## **TOOLS**

## **INTERACT**

This tool lets you interact with the visualized environment. You can click on objects and depending on their properties simply select them, move them around and much more.

## **MOVE CAMERA**

The Move Camera tool is the default tool. When this is selected and you click inside the 3D view, the viewpoint changes according to the options and camera type you have selected in the Views panel

## **SELECT**

The Select tool allows you to select items being displayed in the 3D view. It supports single-point selection as well as click/drag box selection. You can add to a selection with the Shift key, and remove from the selection with the Ctrl key. Alt key to switch back to move the camera.

## **FOCUS CAMERA**

Focus camera lets you select a location in the visualizer. The camera will then focus that point by changing its orientation but not its position.

## **2D POSE ESTIMATE**

This tool lets you set an initial pose to seed the localization system.

## **2D NAV GOAL**

This tool lets you set a goal sent on the goal\_pose ROS topic. Click on a location on the ground plane and drag to select the orientation. The output topic can be changed in tool properties option.

## **PUBLISH POINT**

The publish point tool lets you select an object in the visualizer and the tool will publish the coordinates of that point based on the frame. The results are shown at the bottom like with the measure tool but are also published on the clicked\_point topic.

**TIME :** The Time panel is mostly useful when running in a simulator, since it allows you to see how much ROS Time has passed, vs. how much Wall Clock (aka real) time has passed.

## **BUILDING A CUSTOM RViz DISPLAY**

**RViz Display is a plugin in ROS 2 RViz. It is a base class that provides the framework for creating custom visualization plugins in RViz.**

In **RViz (ROS Visualization)**, different types of messages (like sensor data, robot state, or maps) can be visualized using built-in plugins. However, if your message type is not supported by default, you need an alternative way to display it.

**1)Convert the message to a supported type- (more traffic network and limitations)**

Some message types, like `visualization_msgs/Marker`, are already supported in RViz. If your custom message can be represented using markers (e.g., points, lines, or meshes), you can convert it to a Marker message and visualize it without writing a new plugin.

**Marker messages:** A Marker message in ROS 2 (from the `visualization_msgs` package) is a way to represent and visualize objects in RViz. It allows you to display points, lines, arrows, text, meshes, and more without needing a custom plugin.

`visualization_msgs` is a **ROS message package** that contains message types used for **visualizing data in RViz**. These messages allow you to publish and display different kinds of information

**HEADER FILE-** we need to make a custom header file.

`point_display.hpp` is likely a **C++ header file** that defines a **custom RViz display plugin** for visualizing points in ROS 2. This file is part of a plugin that extends RViz to support custom message types or modify existing visualizations.

The class is templated with our `Point2D` message type.

**you need the `Q_OBJECT` macro in there to get the QT parts of the GUI to work.**

**Qt** is a **cross-platform GUI (Graphical User Interface) framework** used for creating applications with graphical elements.

**Why Do We Need Qt in RViz Plugins?**

When you create a **custom RViz display**, it must integrate with the RViz UI, which is built using Qt.

To achieve this, your custom display class needs the **`Q_OBJECT`** macro, which enables Qt's **signal-slot mechanism** (event handling system).

**What is `Q_OBJECT`?**

`Q_OBJECT` is a macro provided by Qt, used inside a class that inherits from `QObject`. `--processMessage` is the only method that needs to be implemented, which we'll do in the `cpp` file.

IN THE SOURCE FILE, In order for RViz to find our plugin, we need this PLUGINLIB invocation in our code (as well as other things below).

```
<depend>pluginlib</depend>
```

```
<depend>rviz_common</depend>
```

```
<depend>rviz_plugin_tutorial_msgs</depend> #dependencies
```

**pluginlib:** pluginlib is a C++ library in ROS that allows dynamically loading and unloading plugins (modules) at runtime. It helps extend functionality without modifying the core application.

**RViz\_common:** rviz\_common is a ROS 2 package that provides core functionalities of Rviz. It contains classes and tools to create custom displays, panels, and tools for Rviz.

**rviz\_plugin\_tutorial\_msgs:** is a package in ROS 2 tutorials that provides custom message types used in RViz plugin examples.

To generate the proper Qt files, we need to

- Turn CMAKE\_AUTOMOC on.
- Wrap the headers by calling qt5\_wrap\_cpp with each header that has Q\_OBJECT in it.
- Include the MOC\_FILES in the library alongside our other cpp files.

CMAKE\_AUTOMOC is a CMake variable that automatically handles Qt's Meta-Object Compiler (MOC) files in a CMake-based project.

**MOC (Meta-Object Compiler)** is a Qt tool that processes C++ header files containing the Q\_OBJECT macro and generates additional .cpp files needed for Qt's signals, slots, and dynamic properties.

**CMAKE\_AUTOMOC** automates this process, so you don't have to manually call **moc** on every file that requires it.

qt5\_wrap\_cpp is a CMake function that processes Qt header files containing Q\_OBJECT by running Qt's Meta-Object Compiler (MOC) manually.

If CMAKE\_AUTOMOC is OFF (or not working), you can explicitly call qt5\_wrap\_cpp to generate the necessary .moc files.

The macro **pluginlib\_export\_plugin\_description\_file** is used in **CMakeLists.txt** to register a plugin so that RViz can find and load it.

We need to add three lines to the header file:

- `#include <rviz_rendering/objects/shape.hpp>` - There's [lots of options in the rviz\\_rendering package](#) for objects to build your visualization on. Here we're using a simple shape.

## What is rviz\_rendering in ROS 2?

`rviz_rendering` is a **ROS 2 package** that provides **3D rendering utilities** for **RViz plugins**. It is responsible for drawing objects like **shapes, markers, and meshes** inside RViz.

---

## Why is rviz\_rendering Needed?

RViz is built on **Ogre3D**, a 3D graphics engine. However, working directly with Ogre is complex.

**`rviz_rendering` provides high-level utilities** for:

Creating 3D **shapes** (cubes, spheres, cylinders)

Handling **colors, materials, and lighting**

Managing **scene nodes** for placing objects

Displaying **text annotations**

Rendering **lines, arrows, and grids**

### *FOR ACTUAL VISUALIZATION*

we add 1)`#include <rviz_rendering/objects/shape.hpp>`

2)`protected virtual method: void onInitialize() override;`

3)`pointer to our shape object:`

`std::unique_ptr<rviz_rendering::Shape> point_shape_;`

Adding `onInitialize()` - special function in Rviz plugins that get called when display plugin is initialized. The `override` keyword ensures that this method correctly overrides a base class method.

## Why Do We Need to Initialize the RViz Plugin?

When developing an RViz **custom display plugin**, we need to ensure that all the necessary **resources, scene nodes, and objects** are set up **before** the plugin starts receiving and processing messages.

The shape object must be constructed here in the `onInitialize` method rather than the constructor because otherwise `scene_manager_` and `scene_node_` would not be ready.

In RViz, `scene_manager_` and `scene_node_` are part of OGRE 3D, the rendering engine used to display 3D objects in Rviz.

`scene_manager_` is responsible for managing the entire 3D world in Rviz.

`scene_node_` is a transformable object that holds 3D shapes and allows movement within the scene.

## Adding a Pointer to Store the Shape Object

we use `std::unique_ptr` – manages memory automatically. Ensures that only **one instance** of Shape exists at a time. This pointer will hold our shape.

IN THE CPP FILE WE DEFINE THE `ONINITIALIZE()`

```
void PointDisplay::onInitialize()
{
    MFDClass::onInitialize();
    point_shape_ =
        std::make_unique<rviz_rendering::Shape>(rviz_rendering::Shape::Type::Cube, scene_manager_,
        scene_node_);
}
```

MFDClass- alias for `rviz::common::MessageFilterDisplay<T>`

This means that your PointDisplay class **inherits from** `MessageFilterDisplay<Point2D>`.

`MessageFilterDisplay<T>` is a **base class in RViz** used for displaying ROS messages in **real-time**.

It provides built-in functionalities like:

**Handling ROS subscriptions** → Receives incoming messages of type T.

**T** represents the ROS message type that your plugin will subscribe to and visualize.

```
class PointDisplay : public
```

```
rviz_common::MessageFilterDisplay<rviz_plugin_tutorial_msgs::msg::Point2D>
```

Here, `T = rviz_plugin_tutorial_msgs::msg::Point2D`

**Transforming data into RViz's fixed frame** → So objects appear in the correct location.

**Queueing messages when transforms are not available** → Prevents flickering or missing objects.

TO TRANSFORM SCENE\_NODE\_

if we **don't** transform the object, it might appear in the wrong place.

For example:

- If the message is in the **robot's frame (base\_link)**, but RViz is using map, then the object will not appear in the right location.

```
scene_node_->setPosition(position);
```

```
scene_node_->setOrientation(orientation);
```

Extracts x and y values from the message (msg->x, msg->y).

Creates an Ogre::Vector3 object (point\_pos) to store the position.

Moves the shape (point\_shape\_) to this position in RViz.

```
Ogre::Vector3 point_pos;
```

```
point_pos.x = msg->x;
```

```
point_pos.y = msg->y;
```

```
point_shape_->setPosition(point_pos);
```

TO UPDATE DIFFERENT PROPERTIES:

ColorProperty is a class from RViz and is part of the RViz Common Library (rviz\_common). #include <rviz\_common/properties/color\_property.hpp>.

Add in the prototype for updateStyle, which is called whenever the GUI is changed via Qt's SIGNAL/SLOT framework:--ensure that changes made by the user in the RViz UI **immediately** update the visualization

```
std::unique_ptr<rviz_common::properties::ColorProperty> color_property_;
```

This line of code is used in an **RViz display plugin** to store a **color property** that can be modified through the RViz UI.

FOR MORE USER FRIENDLY INTERFACE WE UPDATE THE plugin\_description.xml

| Tag/Field                                 | Purpose                                                                       |
|-------------------------------------------|-------------------------------------------------------------------------------|
| <library path="point_display">            | Specifies the <b>shared library file</b> that contains the plugin.            |
| <class name="Point2D">                    | Changes how the <b>plugin name appears in RViz</b> (more user-friendly).      |
| type="rviz_plugin_tutorial::PointDisplay" | Specifies the actual <b>C++ class</b> that implements the plugin.             |
| base_class_type="rviz_common::Display"    | Defines the base class for this plugin (all RViz display plugins inherit from |

|                |                                                                                                                               |
|----------------|-------------------------------------------------------------------------------------------------------------------------------|
|                | rviz_common::Display).                                                                                                        |
| <message_type> | Associates the plugin with the <b>Point2D message type</b> , so RViz can suggest it when subscribing to a topic of that type. |

In RViz, when adding a new display, you can choose to **add it manually** or **select a display "By Topic."** By Topic" means selecting a display plugin based on the message type of a ROS 2 topic. This allows RViz to automatically suggest a plugin that matches the topic's message type

## BUILDING A CUSTOM RVIZ PANEL

**Include a custom header file:** demo\_panel.hpp

**INHERIT :** rviz\_common::Panel is a **base class** in RViz that allows you to create **custom UI panels** inside the RViz interface.

**explicit DemoPanel(QWidget \* parent = 0);** Ensures the constructor is only called when explicitly used. Constructor with a QWidget\* parent (default: nullptr).

**~DemoPanel() override;** Destructor, cleans up when the object is destroyed. Ensures that this destructor correctly overrides a virtual destructor from the base class (rviz\_common::Panel).

**SOURCE FILE:** In order for RViz to find our plugin, we need this PLUGINLIB invocation in our code (as well as other things below).

```
#include <pluginlib/class_list_macros.hpp>
PLUGINLIB_EXPORT_CLASS(rviz_panel_tutorial::DemoPanel, rviz_common::Panel)
```

On the QT side, we declare a label and a button, as well as a callback for the button (buttonActivated). When working with Qt (which RViz uses for GUI components), we often create **widgets** like labels and buttons. Additionally, we define **callbacks** (or event handlers) to respond to user actions, such as button clicks.

## MARKER:DISPLAY TYPE

The Markers display allows programmatic addition of various primitive shapes to the 3D view by sending a visualization\_msgs/msg/Marker or visualization\_msgs/msg/MarkerArray message.

## MESH RESOURCE

Uses the mesh\_resource field in the marker. Can be any mesh type supported by RViz (binary .stl or Ogre .mesh in 1.0, with the addition of

COLLADA (.dae) in 1.1). The format is the URI-form used by [resource\\_retriever](#), including the package:// syntax.

The mesh\_resource field in a ROS 2 marker allows you to load and display **3D models** (meshes) inside.

### File Formats Supported by RViz:

- **Binary .stl (Stereolithography)**
- **Ogre .mesh** (Used in the OGRE 3D rendering engine)
- **COLLADA .dae (Digital Asset Exchange)** (*Added in RViz 1.1, more flexible for animations and textures*)

The mesh\_resource field **specifies the path** to the mesh file that should be loaded into Rviz.

#### **.stl (Stereolithography)**

- A simple **binary** or **ASCII** file format.
- Stores **only geometry** (no colors, textures, or animations).
- Commonly used in **3D printing** and **CAD software**.

**Example .stl use case:** Visualizing a robot part in Rviz.

#### **.mesh (Ogre Mesh)**

- Used by **OGRE (Object-Oriented Graphics Rendering Engine)**, which RViz relies on for rendering.
- More efficient for rendering complex objects.
- Can include **materials, colors, and textures**.

**Example .mesh use case:** Displaying a **pre-rendered** robot model in Rviz.

#### **.dae (COLLADA - Digital Asset Exchange)**

- A **flexible** XML-based format.
- Can store **animations, textures, materials, and physics properties**.
- Widely supported in **3D modeling tools** (e.g., Blender, Maya, SketchUp).

**Example .dae use case:** Visualizing a **robot arm** with textures and animations

**resource\_retriever** is a ROS package used to **load external files** (e.g., meshes, images, or configurations) from packages or remote URLs.

### How resource\_retriever Works

1. Recognizes the package:// syntax.
2. Finds the ROS package **on the system**.

3. Loads the file **from the correct location**.

The `mesh_use_embedded_materials` flag determines **whether the marker should use colors and textures** from the mesh file itself or override them using the marker's color property.

If the mesh **supports embedded materials** (e.g., COLLADA .dae), then the **textures, colors, and materials** defined inside the mesh file will be **used instead of the marker.color** property.

The **marker** will ignore any colors or textures in the mesh file.